

bdd Contributed Examples (contrib/)

This directory provides practical, ready-to-run examples demonstrating how to use `bdd` as both a **CLI tool** and a **native library (C-ABI and Python)** for real-world binary formats.

```
contrib/
├─ Makefile                # Builds C binaries and executes all tests
├─ README.md              # This documentation
├─ README.pdf             # Print-ready vector PDF documentation
├─ data/                  # Realistic binary sample files
│  └─ sample.mp3          # MPEG-1 Layer III audio frames (128k & 160k)
│  └─ sample.ts           # MPEG-2 Transport Stream (188B packets with PIDs 0, 256, 257)
│  └─ sample.wav          # RIFF WAVE 16-bit stereo PCM audio (44.1 kHz)
│  └─ sample_24bit.raw    # Raw 24-bit studio signed PCM audio
│  └─ sample.mp4          # ISO-BMFF MP4 with ftyp, mdat (H.264 NALs), and moov
│  └─ sample.jpg          # JPEG JFIF with SOF0 (640x480, 4:2:0 subsampling)
│  └─ sample.safetensors  # Safetensors file with FP8, BF16, and FP16 tensors
│  └─ sample_nvfp4.bin    # NVIDIA Blackwell NVFP4 (E2M1) 4-bit packed weights
│  └─ sample_fp6.bin      # OCP Microscaling FP6 (E3M2) 6-bit unaligned weights
│  └─ sample_ipv4.bin     # RFC 791 IPv4 20-byte base packet header
│  └─ sample_udp.bin      # RFC 768 UDP 8-byte datagram header
│  └─ sample_tcp.bin      # RFC 793 TCP 20-byte base segment header (SYN)
│  └─ sample_packets.bin  # Multi-packet capture stream (DNS/UDP, TCP SYN, HTTP GET)
├─ shell/                 # Standalone shell scripts using the bdd CLI
│  └─ decode_mp3.sh       # Unaligned 32-bit MP3 frame header parsing & frame seeking
│  └─ decode_mpeg_ts.sh   # MPEG-TS 13-bit PID inspection & container striding
│  └─ decode_wav.sh       # WAV header parsing, stereo demuxing & 24-bit downsampling
│  └─ decode_mp4.sh       # MP4 box traversal & H.264 NAL bitfield extraction (1U2U5U)
│  └─ decode_jpeg.sh      # JPEG SOF0 geometry & 4-bit chroma nibble extraction (4U4U)
│  └─ decode_ai_weights.sh # Safetensors 0(1) seeking & NVFP4/FP6/FP8/BF16 decoding
│  └─ decode_network.sh   # IPv4, UDP, and TCP header dissection with sub-byte flags
│  └─ run_all_shell_examples.sh # Master script running all shell demonstrations
├─ python/                # Python scripts using libbdd ctypes bindings
│  └─ generate_samples.py  # Deterministic generator for all data/ sample files
│  └─ decode_media.py     # MP3, MPEG-TS, JPEG, and H.264 NAL parsing
│  └─ decode_ai_weights.py # Safetensors inspection, NVFP4, and FP6 decoding
│  └─ decode_network.py   # IPv4, UDP, and TCP packet dissection and IP formatting
├─ c/                     # Native C programs linking against libbdd.so
│  └─ decode_media.c      # bdd_unpack_u64, bdd_pack_u64, and bit reversal
│  └─ decode_ai_weights.c # Native FP4, FP6, FP8, BF16, and FP16 float decoders
│  └─ decode_network.c    # High-performance IPv4/UDP/TCP unpacking with discrete flags
```

Quick Start

Run All Demonstrations

From the repository root:

```
make contrib
```

Or from inside `contrib/`:

```
cd contrib
make test
```

Regenerate Sample Data Files

To recreate all test binary files from scratch:

```
python3 contrib/python/generate_samples.py
```

Format & Bitstream Reference

1. MPEG-1 Layer III Audio (MP3) Frame Headers

Standard Unix tools cannot inspect MP3 frame headers because fields do not align to byte boundaries:

- **Header Bitfield (32 bits = 4 bytes):** 11U (Syncword 0x7FF) 2U (MPEG Version) 2U (Layer) 1U (CRC Protection) 4U (Bitrate Index) 2U (Sample Rate Index) 1U (Padding) 1U (Private) 2U (Channel Mode) 2U (Mode Extension) 1U (Copyright) 1U (Original) 2U (Emphasis)
- **CLI Pattern:**

```
bdd "11U2U2U1U4U2U1U1U2U2U1U1U2U" --count=1 --output-json < sample.mp3
```

- **O(1) Frame Seeking:**

```
# Jump directly to frame 2 at byte 417:  
bdd "417B:32 → 32" "11U2U2U1U4U2U1U1U2U2U1U1U2U" --count=1 --output-json < sample.mp3
```

2. MPEG-2 Transport Stream (MPEG-TS)

MPEG-TS uses repeating 188-byte containers (1504 bits). The 4-byte header contains an unaligned **13-bit Packet Identifier (PID)**:

- **Header Pattern:** Unpack the 32-bit header from each 188-byte packet while discarding the 184-byte payload automatically:

```
bdd "188B[0:32] → 32" 8U1U1U1U13U2U2U4U --count=4 --output-json < sample.ts
```

- **Direct Container PID Extraction:** Extract only the 13-bit PID (bits 11..24) from every 188-byte container with zero manual bitmasking or post-gap math:

```
bdd "188B[11:13] → 13" --output-integers < sample.ts
```

3. WAV / RIFF Audio & Multi-Channel Demuxing

- **Stereo Demuxing:** Skip the 44-byte RIFF header, frame into 32-bit stereo pairs (two 16-bit signed PCM samples), and demux into two independent mono audio files in a single pass:

```
bdd "44B:32 → 32" 16S16S --input-little-endian \  
--demux-files="left.raw,right.raw" < sample.wav
```

- **24-bit PCM Downsampling via Container Slicing:** Directly slice the most significant 16 bits ([0:16]) of each 24-bit PCM sample without needing manual arithmetic shifts:

```
bdd "24[0:16] → 16" 16S --output-tuples < sample_24bit.raw
```

4. MP4 / ISO-BMFF & H.264 NAL Units

- **MP4 Box Walking:** Inspect nested 32-bit big-endian length and 4-byte FourCC ASCII box headers:

```
bdd 32U32C --count=1 --output-json < movie.mp4  
# Or seek directly to box offset:  
bdd "28B:64 → 64" 32U32C --count=1 --output-json < movie.mp4
```

- **H.264 NAL Unit Headers:** Decode 32U (Length), 1U (Forbidden Zero), 2U (NAL Reference IDC), 5U (NAL Unit Type: SPS=7, PPS=8, IDR=5, Slice=1):

```
bdd "36B:40 → 40" 32U1U2U5U --output-json < movie.mp4
```

5. JPEG SOF0 Chroma Subsampling Nibbles

In baseline JPEG (0xFFC0), the horizontal and vertical chroma subsampling factors (e.g., 2×2 for 4:2:0) are stored as **4-bit nibbles** (4U4U):

- **Stream Pattern with Repetition Multiplier:**

```
bdd "24B:120 → 120" "8U16U16U8U3*(8U4U4U8U)" --count=1 --output-json < sample.jpg
```

6. AI Model Weights (Safetensors, NVFP4, FP6, FP8, BF16)

`bdd` natively decodes sub-byte and modern floating-point formats:

CODE	FORMAT	BIT WIDTH	DESCRIPTION
4E / 4e	NVFP4 (E2M1)	4 bits	NVIDIA Blackwell / OCP Microscaling FP4
6E / 6e	FP6 (E3M2)	6 bits	OCP Microscaling FP6
8E / 8e	FP8 (E4M3FN)	8 bits	NVIDIA Hopper H100 / OCP FP8
8Q / 8q	FP8 (E5M2)	8 bits	NVIDIA Ada Lovelace / OCP FP8
16Y / 16y	BF16	16 bits	Google Brain Bfloat16
16H / 16h	FP16	16 bits	IEEE 754 Half-Precision Float

- **Safetensors Inspection:**

```
# 1. Read 64-bit LE JSON header length:
HEADER_LEN=$(bdd "8*8U" --count=1 --output-json < model.safetensors | jq '.[0] + .[1]*256 + .[2]*65536 + .[3]*16777216')
# 2. Seek directly to tensor offset in O(1) time and unpack FP8 weights:
bdd "$(( 8 + HEADER_LEN ))B:8 → 8" 8E --output-json < model.safetensors
```

- **NVIDIA Blackwell NVFP4 (Sub-Byte 4E4E):**

```
bdd 4E4E --output-json < sample_nvfp4.bin
```

- **OCP FP6 (Unaligned 4*6E across 24 bits):**

```
bdd "4*6E" --output-json < sample_fp6.bin
```

7. Network Protocol Headers (IPv4, UDP, TCP)

Network headers are strictly big-endian (network byte order) with MSB-first bit numbering. `bdd` parses these bitfields with natural-order `U` and `B` specifiers, cleanly extracting sub-byte flags without manual bit shifting:

- **RFC 791 IPv4 Base Packet Header (160 bits = 20 bytes):**

- **Preset:** `--preset=ipv4-header`
- **Bitfield:**
`version:4U,ihl:4U,dscp:6U,ecn:2U,total_length:16U,id:16U,flags:3U,frag_offset:13U,ttl:8U,protocol:8U,checksum:16U,src_ip:32U,dst_ip:32U`
- **CLI Command:**

```
bdd --preset=ipv4-header --output-json --json-object < sample_ipv4.bin
```

- **RFC 768 UDP Datagram Header (64 bits = 8 bytes):**

- **Preset:** `--preset=udp-header`
- **Bitfield:** `src_port:16U,dst_port:16U,length:16U,checksum:16U`
- **CLI Command:**

```
bdd --preset=udp-header --output-json --json-object < sample_udp.bin
```

- **RFC 793 TCP Base Segment Header (160 bits = 20 bytes):**

- **Preset:** `--preset=tcp-header`
- **Bitfield:**
`src_port:16U,dst_port:16U,seq_num:32U,ack_num:32U,data_offset:4U,reserved:3U,ns:1B,cwr:1B,ece:1B,urg:1B,ack:1B,psh:1B,rst:1B,syn:1B,fin:1B`
- **Sub-byte Flags:** Every control flag (`ns`, `cwr`, `ece`, `urg`, `ack`, `psh`, `rst`, `syn`, `fin`) is isolated into an individual boolean/integer field.

- **CLI Command:**

```
bdd --preset=tcp-header --output-json --json-object < sample_tcp.bin
```

- **Multi-Packet Capture Stream Dissection:** Combine stream offset patterns (<offset>B:<size> → <size>) with presets or named fields:

```
# Packet 1 (DNS over UDP at offset 0):
bdd --preset=ipv4-header --count=1 --output-json < sample_packets.bin
bdd "20B:64 → 64" --preset=udp-header --count=1 --output-json < sample_packets.bin
bdd "28B:32 → 32" "dns_id:16U,flags:16U" --count=1 --output-json --json-object < sample_packets.bin

# Packet 2 (TCP SYN at offset 32B):
bdd "32B:160 → 160" --preset=ipv4-header --count=1 --output-json < sample_packets.bin
bdd "52B:160 → 160" --preset=tcp-header --count=1 --output-json < sample_packets.bin
```

Native C & Python APIs

C API (`include/bdd.h` & `libbdd.so`)

```
#include "bdd.h"

// 1. Unpack unaligned bitfields (e.g. UDP 64-bit header)
uint64_t udp_fields[4];
bdd_unpack_u64("16U16U16U16U", udp_u64, udp_fields, 4);
// fields: [src_port, dst_port, length, checksum]

// 2. Unpack TCP Word 3 flags and window size in one step
uint64_t tcp_word3[12];
bdd_unpack_u64("4U3U1B1B1B1B1B1B1B1B16U", w3, tcp_word3, 12);
uint64_t syn_flag = tcp_word3[9]; // Isolated SYN bit!

// 3. Bit-exact repacking
uint64_t repacked = 0;
bdd_pack_u64("16U16U16U16U", udp_fields, 4, &repacked);

// 4. Convert AI floating point numbers
double w0 = bdd_decode_fp4_e2m1(0x03); // 1.50
double fp8 = bdd_decode_fp8_e4m3(0x38); // 1.00
double bf16 = bdd_decode_bf16(0x3F80); // 1.00
```

Python API (`python/bdd.py`)

```
from bdd import Bdd

b = Bdd()

# 1. Unpack network datagram headers
src_port, dst_port, length, checksum = b.unpack("16U16U16U16U", udp_u64)

# 2. Extract TCP 32-bit word 3 with discrete control flags
offset, res, ns, cwr, ece, urg, ack, psh, rst, syn, fin, win = b.unpack(
    "4U3U1B1B1B1B1B1B1B1B16U", tcp_w3
)

# 3. Bitfield roundtrip pack
packed_u64 = b.pack("16U16U16U16U", (5353, 53, 12, 0x8899))

# 4. AI float codecs
val_fp4 = b.decode_fp4(0x03)
val_fp8 = b.decode_fp8(0x38)
val_fp6 = b.decode_fp6(0x10)
```